

A Complete tidymodels Workflow

2026-04-17

Introduction

tidymodels is the R meta-package that wraps **rsample**, **recipes**, **parsnip**, **workflows**, **tune**, and **yardstick** into a consistent, tidyverse-friendly interface for machine-learning pipelines. A complete tidymodels workflow composes a data split, a preprocessing recipe, a model specification, hyperparameter tuning on cross-validation resamples, and final evaluation on a held-out test set. The framework's strength is that the same syntax handles regression, classification, and time-series tasks across dozens of underlying engines (lm, glm, random forest, xgboost, neural networks).

Prerequisites

A working knowledge of R basics, the tidyverse, and standard model-fitting concepts (training-test split, cross-validation, hyperparameter tuning).

Theory

The canonical tidymodels pipeline:

1. **Split data** with `rsample::initial_split()` into training and test sets.
2. **Define preprocessing** with **recipes**: imputation, normalisation, dummy encoding, feature engineering.
3. **Specify a model** with **parsnip**, choosing an engine and which hyperparameters to tune.
4. **Combine** into a **workflow** that bundles recipe and model so preprocessing is applied consistently at prediction time.
5. **Tune** hyperparameters on cross-validation resamples (`tune::tune_grid` or Bayesian `tune_bayes`) using a metric set (**yardstick**).
6. **Finalise** on the best hyperparameters and refit on all training data.
7. **Evaluate** on the test set, never touching it before this step.

Assumptions

Data are tidy (one row per observation, one column per variable); roles (outcome vs predictor) are clearly identified; cross-validation splits are appropriate to the problem (random for IID, stratified for imbalance, blocked for time series).

R Implementation

```
library(tidymodels)

set.seed(2026)
df <- tibble(
  x1 = rnorm(500), x2 = rnorm(500),
  x3 = rnorm(500), x4 = rnorm(500),
  y = factor(sample(c("yes", "no"), 500, replace = TRUE,
                    prob = c(0.3, 0.7)))
)
```

```

split <- initial_split(df, strata = y)
train <- training(split); test <- testing(split)
folds <- vfold_cv(train, v = 5, strata = y)

rec <- recipe(y ~ ., data = train) %>%
  step_normalize(all_numeric_predictors())

spec <- rand_forest(trees = 300, mtry = tune(), min_n = tune()) %>%
  set_engine("ranger") %>% set_mode("classification")

wf <- workflow() %>% add_recipe(rec) %>% add_model(spec)

grid <- expand_grid(mtry = c(1, 2, 3), min_n = c(5, 15))
tuned <- tune_grid(wf, folds, grid = grid, metrics = metric_set(roc_auc))

best <- select_best(tuned, metric = "roc_auc")
final_fit <- finalize_workflow(wf, best) %>% last_fit(split)

collect_metrics(final_fit)

```

Output & Results

`tune_grid()` returns a tibble with one row per combination of resample fold and hyperparameter setting, including the metric value (here ROC AUC). `last_fit()` refits the workflow on the full training set with the chosen hyperparameters and evaluates it on the held-out test set in one call.

Interpretation

A reporting sentence: “tidymodels random-forest workflow with 5-fold stratified cross-validation selected `mtry = 2`, `min_n = 5` (best CV ROC AUC 0.81 across the 6-point grid); on the held-out test set the final model achieved ROC AUC 0.79, the honest generalisation estimate.” Always state the resampling scheme, the tuning metric, and the test-set performance.

Practical Tips

- Use `workflow()` rather than direct `fit()` calls; the workflow ensures preprocessing is applied consistently at prediction time without leaking test-set information into transformations.
- `last_fit()` refits on the full training set and evaluates on the test set in a single command; this is the cleanest pattern for the final-model step.
- Stratify splits and CV folds by outcome (`strata = y`) for imbalanced classification; otherwise rare classes can be missing in some folds.
- `hardhat::extract_workflow()`, `extract_fit_engine()`, and `extract_recipe()` retrieve internals from a fitted workflow for inspection or downstream use.
- Parallelise tuning with `doParallel::registerDoParallel(cores = 8)` or the `future` framework; tuning is often embarrassingly parallel across resamples and hyperparameter combinations.
- For Bayesian hyperparameter optimisation, use `tune_bayes()` instead of `tune_grid()`; it converges to the optimum in fewer evaluations on smooth performance landscapes.

R Packages Used

`tidymodels` as the umbrella package; `rsample` for splits and resampling; `recipes` for preprocessing; `parsnip` for unified model specifications; `workflows` for bundling; `tune` for hyperparameter optimisation; `yardstick` for performance metrics; `dials` for hyperparameter ranges; `finetune` for racing-based tuning that prunes poor candidates early.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren't.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis